

KeyPilot

Projektbeschreibung - digitales Schlüsselmanagement

Projektidee: KeyPilot verwaltet Schlüssel, Räume und Mitarbeiter zentral. Jede Schlüsselausgabe und -rückgabe wird nachvollziehbar dokumentiert. Der Schwerpunkt liegt auf Python und PostgreSQL; ergänzend ist eine einfache Webdarstellung mit Flask und HTML vorgesehen.

1. Ausgangssituation

In Unternehmen, öffentlichen Einrichtungen oder größeren Gebäuden müssen häufig viele Schlüssel verwaltet werden. Es soll jederzeit nachvollziehbar sein, welcher Schlüssel zu welchem Raum gehört, wer ihn aktuell besitzt und wann er ausgegeben oder zurückgegeben wurde. Eine zentrale digitale Verwaltung soll unübersichtliche Listen und fehlende Informationen vermeiden.

2. Ziel des Projekts

KeyPilot bildet Mitarbeiter, Schlüssel, Räume und Schlüsselbewegungen in einer relationalen Datenbank ab. Die Kernlogik wird zunächst als CLI-Anwendung umgesetzt. Zusätzlich ist eine bewusst einfache Webdarstellung mit Flask und HTML-Templates vorgesehen. Der Schwerpunkt bleibt auf Backend, Datenbank, sauberer Geschäftslogik und Nachvollziehbarkeit.

3. Geplante Kernfunktionen

- Mitarbeiter, Räume und Schlüssel anlegen, anzeigen und verwalten
- Schlüssel relevanten Räumen zuordnen
- Schlüssel ausgeben und die Ausgabe mit Datum, Mitarbeiter und Schlüssel dokumentieren
- Schlüssel zurücknehmen und die Rückgabe dokumentieren
- Aktuellen Besitzer und Status eines Schlüssels abfragen
- Status wie verfügbar, ausgegeben, verloren, defekt oder ausgemustert verwalten
- Historie bzw. Logbuch aller Schlüsselbewegungen führen
- Anzeigen, welche Schlüssel ein Mitarbeiter aktuell besitzt oder früher besessen hat

4. Daten und Datenbank

Als Datenbanksystem ist PostgreSQL vorgesehen. Neben den Stammdaten wird insbesondere die Historie der Ausgaben und Rückgaben gespeichert.

Bereich	Geplante Testdaten
Schlüssel	ca. 80
Räume	ca. 50
Mitarbeiter	ca. 30
Reinigungskräfte	ca. 5

5. Datenmodell / Entitäten

Das Datenmodell wird bewusst übersichtlich gehalten. Reinigungskräfte werden nicht als eigene Entität geführt, sondern als Mitarbeiter mit entsprechender Rolle bzw. Mitarbeiterart. Die endgültigen Felder können während der Modellierung noch angepasst werden.

Entität	Wichtige Datenpunkte
Mitarbeiter	ID, Personalnummer, Vorname, Nachname, Abteilung/Bereich, Rolle bzw. Mitarbeiterart, Status aktiv/inaktiv
Raum	ID, Raumnummer, Bezeichnung, Gebäude/Standort, Status bzw. gesperrt/freigegeben
Schlüssel	ID, Schlüsselnummer, Bezeichnung, zugeordneter Raum bzw. zugeordnete Räume, aktueller Status
Schlüsselbewegung / Ausgabe	ID, Mitarbeiter, Schlüssel, Ausgabedatum/-zeit, Rückgabedatum/-zeit, Art der Bewegung und Status

6. Technische Umsetzung

- Programmiersprache: Python
- Datenbank: PostgreSQL
- Kernanwendung: CLI / Terminal
- Webdarstellung: Flask und einfache HTML-Templates, zunächst ohne CSS
- Versionsverwaltung: Git und GitHub
- Architektur: Trennung von Programmlogik, Datenbankzugriff und Benutzerschnittstellen

7. Einfache Webdarstellung

Die Weboberfläche soll dieselben Daten und dieselbe Geschäftslogik wie die CLI verwenden. Für die erste Version reichen einfache HTML-Seiten ohne aufwendiges Design.

- Übersicht der Mitarbeiter
- Übersicht der Schlüssel und ihres aktuellen Status
- Anzeige der aktuell ausgegebenen Schlüssel
- Detailansicht eines Mitarbeiters mit seinen aktuellen Schlüsseln
- Optional einfache Formulare für Ausgabe und Rückgabe

8. Dokumentation

Zum Projekt gehört neben dem Quellcode eine verständliche Dokumentation. Sie soll sowohl die technische Umsetzung als auch die Nutzung von KeyPilot nachvollziehbar machen.

- README: Projektziel, Voraussetzungen, Installation und Start der Anwendung
- Datenmodell: Beschreibung der Entitäten, Beziehungen und wichtigsten Datenfelder
- Bedienung: kurze Beschreibung der wichtigsten CLI- und Webfunktionen
- Technische Dokumentation: Projektstruktur, Datenbankbindung und zentrale Programmlogik
- Fachliche Nachvollziehbarkeit: Ausgabe, Rückgabe und Historie der Schlüsselbewegungen werden durch die Anwendung selbst dokumentiert

9. Mögliche Zusatzfunktionen

Die folgenden Funktionen gehören nicht zum notwendigen Kernumfang und können umgesetzt werden, wenn ausreichend Projektzeit vorhanden ist.

- Rückgabefristen für ausgegebene Schlüssel festlegen und anzeigen
- Rückgabefristen verlängern und Verlängerungen dokumentieren
- REST-API für externe Abfragen
- PDF-Ausgabe eines Übergabe- bzw. Rückgabeprotokolls
- CSV-Import und CSV-Export
- Administrative Funktionen zum Sperren von Räumen oder Ausmustern von Schlüsseln
- Unterstützung des Rückgabeprozesses beim Ausscheiden eines Mitarbeiters

10. Geplanter Projektablauf

- 1. Anforderungen und Datenmodell festlegen
- 2. PostgreSQL-Datenbank und Tabellen erstellen
- 3. Testdaten erzeugen
- 4. CRUD-Funktionen für Stammdaten implementieren
- 5. Ausgabe und Rückgabe inklusive Dokumentation implementieren
- 6. Historie, Statusabfragen und Validierungen ergänzen
- 7. CLI fertigstellen und testen
- 8. Einfache Flask-/HTML-Darstellung ergänzen
- 9. Projektdokumentation fertigstellen und bei verfügbarer Zeit Zusatzfunktionen umsetzen

11. Ergebnis

Am Ende soll eine funktionsfähige Backend-Anwendung vorliegen, mit der sich ein realistischer Schlüsselbestand strukturiert verwalten lässt. Jede Ausgabe und Rückgabe wird nachvollziehbar dokumentiert. Die einfache Webdarstellung zeigt zusätzlich, dass dieselbe Programmlogik und Datenbank über einen Browser genutzt werden kann. Das Datenmodell und die Projektdokumentation machen Aufbau und Nutzung des Systems nachvollziehbar.